

Cine IQ Project Report

1. Introduction

Cine IQ addresses the content discovery problem faced by users on modern streaming platforms. Recommendation feeds can become opaque, biased toward promoted titles, and repetitive over time. The goal of this project is to create an open and explainable movie recommendation engine that combines multiple machine learning strategies while still presenting recommendations in a way users can understand.

The system is built around a hybrid recommendation pipeline. Collaborative filtering captures user-to-user and user-to-item preference patterns from MovieLens ratings. Content-based filtering uses metadata such as genres, keywords, cast, director, and overview text to identify similar movies. A weighted ensemble combines both signals, while a sentiment-aware re-ranker adjusts the final order using audience reception signals from review sentiment. Finally, the explainability layer provides readable explanations for each recommendation, helping users see why a title was suggested.

2. Objectives and Requirements Coverage

The Cine IQ assignment asks for a hybrid recommendation engine, a sentiment-aware re-ranker, a user taste dashboard, an explainability layer, and a serving stack using FastAPI, Streamlit, Plotly, and MLflow. The current codebase covers these major components:

Requirement	Coverage in the Codebase
Hybrid recommendation engine	<code>HybridEnsemble</code> combines SVD collaborative predictions and TF-IDF content scores using configurable weights.
Collaborative filtering	<code>CollaborativeFilter</code> trains a Surprise SVD model and exposes prediction and top-N methods.
Content-based filtering	<code>ContentBasedFilter</code> builds a TF-IDF matrix over movie metadata and computes cosine similarity.
Sentiment-aware re-ranking	<code>SentimentReRanker</code> supports VADER and DistilBERT scoring and adjusts recommendation scores using a gamma parameter.
User taste dashboard	The Streamlit dashboard includes recommendation, similar movie, genre, decade, and director preference views.
Explainability	The explainer generates rule-based recommendation reasons and can include LIME-derived text features.
Serving	FastAPI exposes <code>/recommend</code> , <code>/similar</code> , and <code>/health</code> ; Streamlit provides the interactive interface.
Tracking	MLflow is used in collaborative model training and ensemble weight tuning.

Overall, the implementation is aligned with the expected Cine IQ feature set. The main remaining dependency for a live final demo is the availability of raw datasets and trained local model artifacts, which are intentionally not stored in GitHub because of size and licensing constraints.

3. System Design

The data pipeline begins with three sources: MovieLens 25M for ratings, TMDb metadata for movie descriptions and people metadata, and IMDb review data for sentiment experiments. `preprocess.py` cleans sparse ratings, joins MovieLens movies to TMDb through `tmbdId`, parses JSON-like metadata fields, extracts directors, and builds a

combined text field called soup. This soup becomes the feature text for the content-based model.

The collaborative model uses Surprise SVD. It learns latent user and movie factors from rating history, evaluates using RMSE and MAE, logs runs to MLflow, and saves the trained model as a pickle file. For recommendations, it predicts ratings for movies a user has not already rated and returns the highest-scoring candidates.

The content model uses TfidfVectorizer with unigrams and bigrams over the metadata soup. Cosine similarity is computed efficiently with a linear kernel. The model can return movies similar to a given title and can also estimate a content score for recommendation candidates based on the user's liked movies.

The hybrid ensemble first asks the collaborative model for candidate movies. It then normalizes collaborative and content scores, blends them with alpha and beta, and returns a ranked list. This design gives the collaborative model responsibility for personalization while the content model adds explainable metadata similarity.

The sentiment module can load precomputed movie sentiment scores or compute scores from raw reviews. It applies a final score formula:

```
final_score = hybrid_score * (1 + gamma * sentiment_score)
```

Positive audience sentiment can move a film slightly upward, while negative sentiment can reduce its rank. This is intentionally a nudge rather than a replacement for personalization.

4. User Interface and API

The FastAPI service exposes three endpoints. `/health` verifies that the service is running. `/recommend` returns personalized recommendations with scores and explanations. `/similar` returns content-similar movies for a given movie ID. This makes the model layer usable by external clients.

The Streamlit dashboard is the main user-facing interface. It includes three workflows: getting personalized recommendations, finding similar movies, and viewing a user taste profile. The taste profile view visualizes genre preferences, decade preferences, director affinity, and rating distribution with Plotly charts.

5. Evaluation Plan

The collaborative model should be evaluated with RMSE and MAE on a held-out ratings split. The hybrid ensemble should be evaluated with Precision@10 during weight tuning, using highly rated movies as the relevance proxy. Sentiment re-ranking can be evaluated by measuring rank shifts and checking whether positive sentiment improves recommendation ordering without overpowering personalization. The API and dashboard should be smoke-tested after model training to confirm model loading and expected response formats.

6. Limitations and Future Work

The major practical limitation is data packaging. MovieLens 25M, TMDB metadata, and IMDb review data should be downloaded separately by the evaluator. Trained model artifacts are also generated locally and excluded from Git. This is normal for a project of this size, but the README must clearly document the setup steps.

The sentiment component is also limited by the common IMDb 50K review dataset because it does not always contain stable movie identifiers. A production version should use review data with IMDb IDs or TMDB IDs so sentiment scores can be joined reliably. Future work could add cold-start onboarding, richer user feedback loops, poster artwork, authentication, and scheduled model retraining.

7. Conclusion

Cine IQ successfully implements the required structure for an explainable hybrid movie recommender. It combines collaborative filtering, content similarity, weighted ensembling, sentiment re-ranking, explainability, an API, a dashboard, and experiment tracking. With datasets downloaded and models trained, the project is ready to be demonstrated as a complete Cine IQ submission.